

Topic: Backpropagation in CNN (Part 2)

(How gradients flow through Convolution, MaxPooling, and Flatten layers)

Introduction

In the previous lesson (Part 1), we understood:

- What backpropagation means
- How it updates weights and biases
- The general forward + backward flow of CNN

Now, we'll go deeper — learning **how** CNN handles gradients *layer by layer*.

Overview: Backpropagation in CNN

When a CNN trains, each layer performs two main operations:

1. **Forward Pass:**
 - Compute outputs (feature maps, activations, etc.)
2. **Backward Pass:**
 - Compute **gradients** (how each weight affects loss)
 - **Update** the weights using these gradients

We apply this principle separately for:

- Convolution Layers
 - MaxPooling Layers
 - Flatten Layers
-

Recap: Backpropagation in CNN

Backpropagation means:

Propagating the **error (loss gradient)** backward through all layers to adjust weights.

It's based on the **chain rule** from calculus:

$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial Z} \cdot \frac{\partial Z}{\partial W}$$

So, to know how much to change each weight (W),

we find how the loss (L) changes when (Z) (the output of that weight) changes.

Applying Gradients

During backprop, we use the gradients calculated at each step to:

1. **Compute derivatives** of the loss function with respect to layer outputs.
 2. **Propagate them backward** through each layer.
 3. **Update weights and biases** (using Gradient Descent).
-

So the full cycle is:

Forward: Input → Conv → Pool → Flatten → Dense → Output

Backward: Output → Dense → Flatten → Pool → Conv → Input

Each layer has its own *backward rule* for handling gradients.

Changing w_1 affects Z_1

To understand the idea, let's focus on one neuron:

$$Z_1 = W_1 \times X_1 + W_2 \times X_2 + b$$

If we change W_1 , then Z_1 changes too —

and since Z_1 affects the loss, we can find:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial Z_1} \times \frac{\partial Z_1}{\partial W_1}$$

In a CNN, Z_1 is not a single number but an **activation map**,
so each small weight in the filter contributes to multiple outputs.

That's why CNN backprop involves **convolutions** and **deconvolutions** during gradient flow.

Backpropagation in MaxPooling Layers

Pooling layers don't have learnable weights.

But we still need to **send the error back** to the correct input pixels.

Let's understand this simply.

Forward Pass (MaxPooling):

- Divide the input into small windows (like 2×2 blocks)
- Take the **maximum** value from each window

Example:

Window Output

[1, 5, 3, 2] 5

So, in forward pass → output = 5 (maximum).

Backward Pass:

When we get the gradient from the next layer (say 1.0),
we only send that gradient **back to the position of the max value**.

Input	Gradient
1	0

5	1
3	0
2	0

Thus, only the “winner” in each pooling window gets the gradient — the rest get zero.

This ensures the network adjusts only where the “important” feature was.

Mathematical Idea for MaxPooling Backprop

We can describe it using a **mask** (a binary matrix):

- 1 → position of the max value
- 0 → everything else

So during backprop:

$$\delta_{input} = \text{mask} \times \delta_{output}$$

where δ = gradient (error signal).

That’s how pooling passes gradients backward.

Backprop in Convolution + Pooling + Flatten

Now let’s combine them all.

1. Convolution Layer:

- Forward: ($Y = X * W + b$)
- Backward:
 - Gradient wrt weights:

$$\frac{\partial L}{\partial W} = X * \frac{\partial L}{\partial Y}$$

- Gradient wrt input:

$$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} * W_{\text{rotated}}$$

[Here “rotated” means flipping the kernel 180° — the *deconvolution* step.]

(Here “rotated” means flipping the kernel 180° — the *deconvolution* step.)

This lets us push the gradient from output → input, showing which pixels influenced the loss.

2. MaxPooling Layer:

- Gradient flows only to the maximum element in each window (as explained).

3. Flatten Layer:

- The flatten layer just reshapes the data from 3D → 1D before the dense layer.
- During backprop, it simply reshapes the gradient from 1D → 3D again.
No weights here — it just passes gradients backward unchanged.

Expanding and Differentiating

For convolution backprop, we sometimes need to expand gradient expressions to match input shape.

This can involve:

- **Padding** the gradients
- **Flipping** the filters (rotating them 180°)
- **Summing** over channels if multiple filters exist

The main goal is to correctly assign the loss contribution to every pixel and weight.

Deconvolution and Multiplication

The term **deconvolution** (or “transposed convolution”) comes into play because:

- Forward convolution reduces spatial size (due to kernel movement).
- Backprop needs to “expand” gradients back to the original size.

So we reverse the operation using **deconvolution**:

$$\delta_X = \delta_Y * W_{\text{rotated}}$$

This tells the model how to adjust each input pixel’s contribution to the error.

In Summary: How Backprop Works Layer by Layer

Layer	What It Learns	How Backprop Works
Convolution Layer	Learns filters (feature detectors)	Gradient w.r.t. filters and inputs via convolution & deconvolution
MaxPooling Layer	No learning; only feature selection	Pass gradient only to max positions using a mask
Flatten Layer	Just reshapes data	Reshape gradients to previous shape

Key Takeaways

- **Backpropagation** adjusts filters so they detect features more accurately over time.
- **MaxPooling** passes gradients only to dominant features (max positions).
- **Flatten** only reorganizes data for dense layers.
- **Convolution backprop** uses both convolution (for weight gradients) and deconvolution (for input gradients).

In Simple Words

CNN backpropagation is like tracing your steps backward — figuring out which pixels, filters, or neurons caused a mistake and fixing only those parts.

Each layer contributes differently:

- Convolution learns *what* to look for
 - Pooling decides *where* to look
 - Flatten connects everything to *predict the right answer*
-